

How the Internet Delivers a Web Page

Website Analyzed: <https://www.google.com>

1. The TCP/IP Protocol Stack

The TCP/IP suite is a layered framework designed to provide internetworking. For our browser to fetch *Google.com*, each of the five layers performs a critical distinct role:

- **Application Layer:** The highest layer where protocols like HTTP or DNS reside. It defines how the browser and server "talk" to exchange the webpage content.
- **Transport Layer:** Uses TCP to ensure data packets reach the destination in the correct order and without errors (Process-to-Process communication).
- **Network Layer:** Uses the IP protocol to route packets through various routers across the global Internet based on the server's IP address.
- **Data Link Layer:** Responsible for transmitting packets over a specific physical link (e.g., from your laptop to your Wi-Fi router).
- **Physical Layer:** Concerns the hardware level—sending bits as electrical, optical, or radio signals.

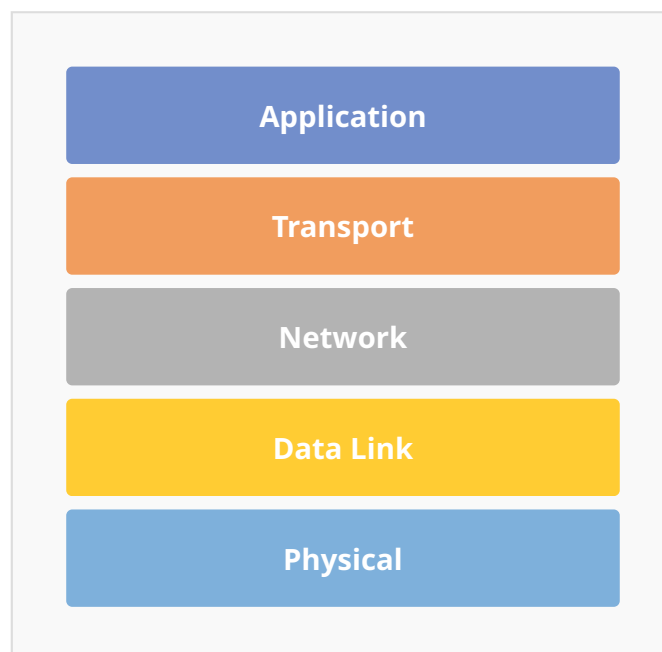


Figure 1: The 5-Layer TCP/IP Protocol Stack

2. Role of DNS in Address Conversion

Computers identify each other via numeric IP addresses, but humans prefer names like "google.com". The **Domain Name System (DNS)** bridges this gap. When you press Enter, the browser sends a query to a DNS server. This server looks up its hierarchical database and returns the IP address associated with Google's servers. This is known as *Name-to-Address resolution*, similar to looking up a contact in a phonebook before dialing.

3. Requesting the Webpage via HTTP

With the IP address identified, the browser initiates the **HyperText Transfer Protocol (HTTP)**. It sends an *HTTP GET Request*. This text-based message includes headers that specify the host (google.com) and tell the server what type of content the browser can display (HTML, CSS, images). The server processes this and returns an *HTTP Response* containing the code for the homepage.

4. Reliable Communication with TCP

HTTP is a "stateless" protocol and doesn't handle transmission errors; it relies entirely on **TCP** for reliability. TCP ensures "Byte-Stream Service" through:

- **Connection Establishment:** A 3-way handshake to sync sequences.
- **Error Checking:** If a packet is lost due to network congestion, TCP detects it and retransmits the data.
- **Flow Control:** Ensuring Google doesn't send data faster than your device can process it.

5. Architecture Analysis: Client-Server

This interaction follows the **Client-Server Paradigm**.

Justification: In this model, the role and lifetime of each program differ. Google hosts a **Server Process** that runs infinitely on powerful hardware, waiting for incoming requests. The browser is a **Client Process** that is finite—it starts when you type the URL, initiates communication, receives the data, and then displays the result. There is a clear hierarchy; the client always initiates, and the server always responds.

6. Comparison: HTTP vs FTP

While both are standard application-layer protocols, they serve different purposes:

Difference	HTTP (HyperText Transfer Protocol)	FTP (File Transfer Protocol)
Primary Goal	Optimized for fetching hypermedia documents (webpages).	Designed for general transfer of files between stores.
Connections	Typically uses one TCP connection to fetch a page.	Uses two parallel connections: one for control and one for data.
Port Usage	Standard port is 80 (HTTP) or 443 (HTTPS).	Port 21 for Control commands and Port 20 for Data transfer.
Statefulness	A stateless protocol; each request is independent.	Stateful; it remembers current directories and user login info.

7. Improving User Experience via Cookies

Because HTTP is stateless, the server "forgets" the user as soon as the response is sent. **Cookies** solve this by allowing the server to store a small file on the client's browser.

Real-World Example: On a site like Amazon, cookies store your "Shopping Cart" ID. When you add an item, the server sends a unique ID to your browser. As you browse other pages, your browser sends that ID back, allowing the server to retrieve your specific cart items, providing a seamless, personalized experience without requiring constant re-authentication.

8. Complete Communication Flow Diagram

This diagram illustrates the sequence of events from URL input to the final webpage display:

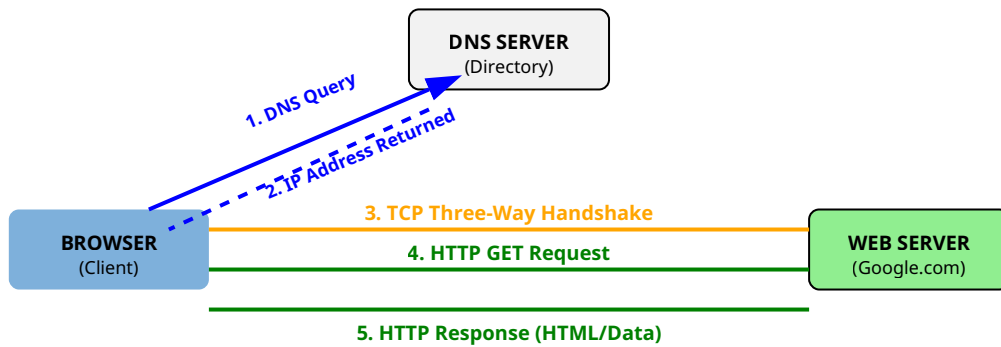


Figure 2: Sequence of DNS Resolution, TCP Handshake, and HTTP Transaction

9. Conclusion

Application layer protocols are the fundamental "language" of the modern Internet. While lower layers are responsible for the complex logistics of moving bits across wires and routers, application protocols like HTTP and DNS define how we interact with information. They abstract the complexities of the network, allowing users to simply type a name and receive content. DNS acts as the Internet's essential directory, while HTTP manages the structured exchange of human-readable data. Without these standardized rules, the Internet would remain a technical network for experts rather than a global platform for commerce, social interaction, and information sharing. Their flexibility allows for the continuous evolution of the Web while ensuring that the simple act of visiting a website remains fast, reliable, and user-friendly in our daily lives.